Table of Contents

Circular Dataset Structures (Matlab only) 1
Divergence from home direction 2
Fit the model 3
Test the expected angle hypothesis 4
Change in direction 5
Test the direction change hypothesis 7
Change in concentration 9
Test the concentration change hypothesis 10
High inter-individual correlation 12
Low inter-individual correlation 14
Test shared heading bias hypothesis 17
Variable individual parameters 18
Individual heading changes 21
Test hypothesis of treatment effect 25
Plot model predictions 27
Summary 29

Circular Dataset Structures (Matlab only)

Matlab version of "R/Circular Dataset Structures.Rmd": simulated circular datasets that provide examples of effects on mean direction, concentration, and individual differences. Fitted circular models are used to estimate these effects and compare hypotheses about their causes.

This uses the same strategy as MatlabOnly.m: the log posterior of the unwrap model (circular modulo link for μ, softplus link for κ) is written out explicitly and sampled with Hamiltonian Monte Carlo (`hmcSampler`, Statistics and Machine Learning Toolbox). No Stan installation is needed.

Differences from the R version:

- gradients are analytic (finite differences are too slow for the hierarchical models)
- leave-one-out cross-validation uses Pareto-smoothed importance sampling (PSIS-LOO), with exact refits for observations with Pareto $k > 0.7$ (equivalent to `loo(..., reloo = TRUE)`, instead of moment matching)
- posterior densities of mean vectors are shown as clouds of posterior draws
- Rhat is the classic (not rank-normalised) version used in MatlabOnly.m; summary tables show it both before (`Rhat_raw`) and after unwrapping circular parameters (`Rhat`)

Inputs: `circular_datasets/*.csv`, exported by "R/Export Circular Datasets.R", so that all versions model identical simulated data, and the functions in `unwrap_functions/`.

Set `saveFigures = true` before running to save figures to `figures_circular/`. Set `quickTest = true` for a fast check with short chains.

```
if ~exist('saveFigures', 'var'); saveFigures = false; end
scriptDir = fileparts(mfilename('fullpath'));
addpath(fullfile(scriptDir, 'unwrap_functions'));
dataDir = fullfile(scriptDir, 'circular_datasets');
figDir = fullfile(scriptDir, 'figures_circular');
```

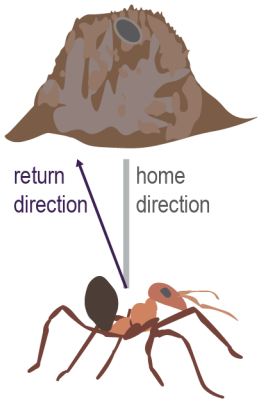
```
if saveFigures && ~exist(figDir, 'dir'); mkdir(figDir); end

rng(120810506) % ISBN Batschelet 1981

% Sampler settings (as for brms defaults: 4 chains, 1000 warmup, 1000 draws)
opts = struct('numChains', 4, ...
              'numSamples', 1000, ...
              'burnin', 1000, ...
              'targetAccept', 0.65, ...
              'reloo', true, ... % exact refits for observations with high
Pareto k
              'maxRefits', 10); % if more observations have high k, report
them instead
if exist('quickTest', 'var') && quickTest
    opts.numSamples = 200; opts.burnin = 200; opts.reloo = false;
end

% Colours
col.kappa = hex2rgb('#1E78B5'); % kappa parameter
col.rho = hex2rgb('#F08024'); % mean vector length
col.sd = hex2rgb('#E74A29'); % SD and mean angle
col.sd2 = hex2rgb('#E57461'); % other SD heuristics
col.obs = hex2rgb('#3E1F51'); % control observations
col.treat = hex2rgb('#006400'); % treatment observations
```

Divergence from home direction



Desert ants of the genus Cataglyphis are known for their use of path integration (dead reckoning) to return to their nest. Disruption of visual orientation cues can result in deviations from the expected homewards direction.

In navigation experiments, return directions are often compared with the expected home direction to determine whether homeward navigation is perturbed. Here, the example data ($n = 20$) were generated from a distribution with a mean that differs from the expected direction (0°) by only 15° ($\kappa = 10$).

A common short-cut is the v-test modification of the Rayleigh test, which tests uniformity against clustering in some direction less than 90° from the expected direction. The data are very far from uniform and the mean angle is close to the expected angle, so the test is significant. Even so, the data show a trend away from the expected angle, with 75% of points falling to the left of 0°.

```
dtDiv = readtable(fullfile(dataDir, 'divergence.csv'));
yDiv = dtDiv.y;
fprintf('Proportion left of 0°: %.2f\n', mean(wrapPi(yDiv) < 0));
[vStat, vP] = vTest(yDiv, 0);
fprintf('v-test (expected direction 0°): V = %.3f, p = %.2g (normal
approximation)\n', vStat, vP);
```

```
Proportion left of 0°: 0.75
v-test (expected direction 0°): V = 0.950, p = 9.3e-10 (normal approximation)
```

Fit the model

Each of μ and κ requires a single parameter estimate (an intercept). We expect a mean direction anywhere on the circle and high concentration ($\kappa > 1$), so highest prior density is set in that range: $\mu \sim N(0, 2\pi)$ and, on the inverse-softplus scale, $\kappa \sim N(3, 3)$. Since the mean direction is far from the wrap-point ($\pm\pi$), Rhat for μ can be interpreted on the linear scale.

```
mDiv = makeModel(yDiv, ...
    'Xmu', ones(numel(yDiv), 1), 'muNames', {'b_Intercept'}, ...
    'muPrior', [1 0 2*pi], ... % normal(0, 2pi)
    'Xk', ones(numel(yDiv), 1), 'kNames', {'b_kappa_Intercept'}, ...
    'kPrior', [3 3]); % normal(3, 3)
fitDiv = fitUnwrapModel(mDiv, opts, 'model_divergence');
printSummary(fitDiv);
```

```
Fitting model_divergence (2 parameters)... 2 s, acceptance ratio [0.89 0.91
0.92 0.93]

model_divergence (circular parameters unwrapped, radians)
          Parameter             Mean          SD          Q2_5
Q97_5        Rhat       Rhat_raw     ESS
    ________________________    _________    _________    _________
_________    ________    _________    ____

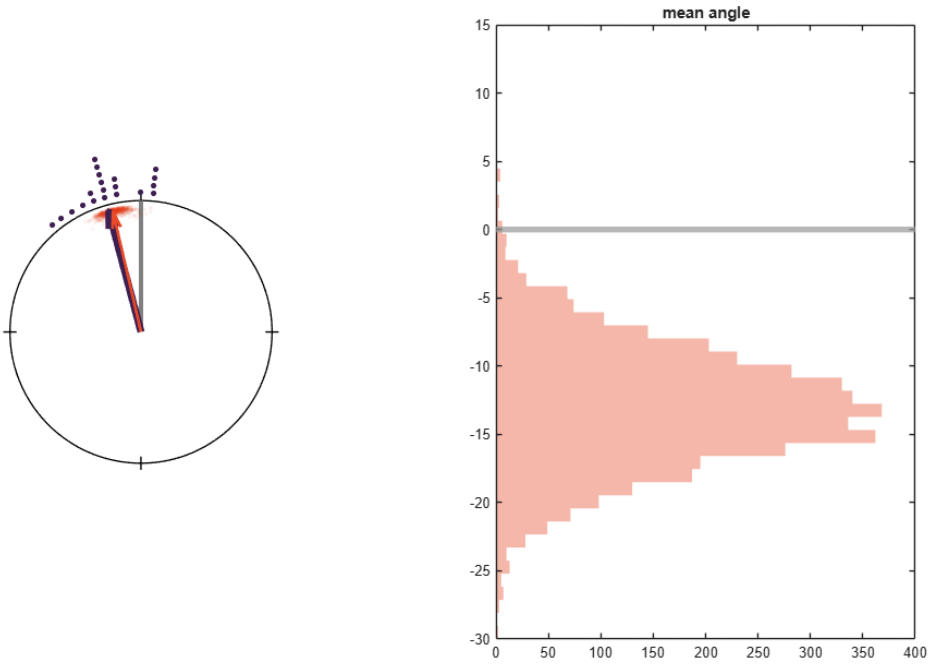
    {'b_Intercept'       }    -0.22802    0.077165    -0.37954
-0.076037    0.99976    0.99976      3406
    {'b_kappa_Intercept'}      8.9989      1.9976      5.1899
13.141     1.0002     1.0002      3029
```

The posterior estimates are summarised as the median mean vector (red arrow), compared with the true mean vector (dark purple arrow), and a cloud of posterior mean vectors.

```
figure('Name', 'Divergence model', 'Color', 'w');
tiledlayout(1, 2);
ax = nexttile; circBase(ax, 0);
circPoints(ax, yDiv, col.obs);
drawCloud(ax, getDraw(fitDiv, 'b_Intercept'), softplus(getDraw(fitDiv,
'b_kappa_Intercept')));
circArrow(ax, deg2rad(-15), A1(10), col.obs, 5);
circArrow(ax, circMedian(getDraw(fitDiv, 'b_Intercept')), ...
    A1(softplus(median(getDraw(fitDiv, 'b_kappa_Intercept')))), col.sd, 2);
ax = nexttile;
vertHist(ax, rad2deg(unwrapCircular(getDraw(fitDiv, 'b_Intercept'))), ...
```

```
    'mean angle', [-30 15], col.sd, 0);
saveFig(saveFigures, figDir, 'divergence_model');
fprintf('mu left of expected angle: %.1f%% of estimates\n', ...
    100*mean(wrapPi(getDraw(fitDiv, 'b_Intercept')) < 0));
```

```
mu left of expected angle: 99.7% of estimates
```



The fitted model indicates that the mean direction is to the left of the expected direction in $> 99\%$ of μ estimates.

Test the expected angle hypothesis

To test the hypothesis that the true mean direction falls in the expected direction, we fit another model in which the mean direction is fixed at the expected direction (no μ parameter), and compare the two models.

```
mExpect = makeModel(yDiv, ...
    'Xk', ones(numel(yDiv), 1), 'kNames', {'b_kappa_Intercept'}, 'kPrior',
[3 3]);
fitExpect = fitUnwrapModel(mExpect, opts, 'model_expect');
looDiv = looUnwrap(fitDiv, opts);
looExpect = looUnwrap(fitExpect, opts);
looCompare({'model_divergence', 'model_expect'}, {looDiv, looExpect});
fprintf('ELPD difference (divergence - expected): %s\n', elpdDiff(looDiv,
looExpect));
```

```
Fitting model_expect (1 parameters)... 1 s, acceptance ratio [0.93 0.89 0.91
0.93]

LOO comparison
           Model             elpd_diff    se_diff      elpd       se_elpd
```

```
    ____________________    _________    _______    _______    _______

    {'model_divergence'}          0          0      -2.2859    1.3279
    {'model_expect'    }     -4.2197     1.6944      -6.5056    2.1806

ELPD difference (divergence - expected): 4.2 ± 1.7
```

The model for a divergent mean direction has higher expected log predictive density (ELPD) than the model for the expected direction. This ELPD difference is $> 2\times$ larger than its standard error, so we can state with some confidence that the hypothesis of a divergence from the expected direction is more likely than the hypothesis of alignment with the expected direction.

If the expected direction is not 0°, we align the data with the expected angle, then fit both models (expected direction and divergent direction) to the aligned data. Here is an example with the expected angle at -15°.

```
yAligned = yDiv - deg2rad(-15);
fitTrue = fitUnwrapModel(setY(mExpect, yAligned), opts, 'model_true');
fitFalse = fitUnwrapModel(setY(mDiv, yAligned), opts, 'model_false');
sFalse = printSummary(fitFalse, false);
fprintf('model_false mu estimate (deg; mean, SD, 2.5%%, 97.5%%): %s\n', ...
    mat2str(round(rad2deg(table2array(sFalse(1, {'Mean', 'SD', 'Q2_5',
'Q97_5'})))))));
looTrue = looUnwrap(fitTrue, opts);
looFalse = looUnwrap(fitFalse, opts);
looCompare({'model_true', 'model_false'}, {looTrue, looFalse});
fprintf('ELPD difference (expected at -15° - divergent): %s\n',
elpdDiff(looTrue, looFalse));
```

```
Fitting model_true (1 parameters)... 1 s, acceptance ratio [0.84 0.9 0.91
0.9]
Fitting model_false (2 parameters)... 1 s, acceptance ratio [0.92 0.91 0.87
0.91]
model_false mu estimate (deg; mean, SD, 2.5%, 97.5%): [2 4 -7 11]

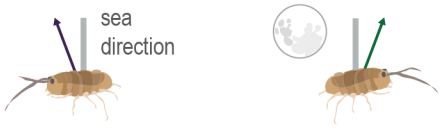
LOO comparison
         Model          elpd_diff    se_diff     elpd      se_elpd
    ________________    _________    _______    _______    _______

    {'model_true' }           0          0     -1.5648    1.2663
    {'model_false'}     -0.72079     0.34151    -2.2855    1.3288

ELPD difference (expected at -15° - divergent): 0.7 ± 0.3
```

Now the expected direction model has higher predictive power, even though both find a similar mean direction. If the divergence model had higher predictive power, but within one standard error of the expected direction model, we could not rule out the possibility that the expected direction is the true direction, though evidence would be weaker.

Change in direction



Sandhoppers (Talitridae) forage along beaches, following the movement of the sea to avoid drying out. Their movement is guided by celestial cues, including the moon.

If the expected direction is not known, it can be estimated from control data and compared with treatment data. In this example, the treatment induces a rightward turn of 30°, changing μ from 180° to 210°, with intermediate concentration ($\kappa = 3$).

The treatment indicator x is used as a predictor of both μ and κ. Predictors are not centred, so the intercepts describe the control condition (equivalent to `0 + Intercept` in `brms`), and the `x` coefficients describe the change caused by the treatment. For stability, a slightly narrower prior on the intercept, $N(0, 3\pi/2)$, is used.

```
dtDelta = readtable(fullfile(dataDir, 'delta.csv'));
nD = height(dtDelta);
mDelta = makeModel(dtDelta.y, ...
    'Xmu', [ones(nD, 1) dtDelta.x], 'muNames', {'b_Intercept', 'b_x'}, ...
    'muPrior', [1 0 3*pi/2; 1 0 pi/2], ...
    'Xk', [ones(nD, 1) dtDelta.x], 'kNames', {'b_kappa_Intercept',
'b_kappa_x'}, ...
    'kPrior', [3 3; 0 3]);
fitDelta = fitUnwrapModel(mDelta, opts, 'model_delta');
printSummary(fitDelta);
```

```
Fitting model_delta (4 parameters)... 3 s, acceptance ratio [0.92 0.93 0.93
0.94]

model_delta (circular parameters unwrapped, radians)
          Parameter             Mean        SD          Q2_5        Q97_5
Rhat      Rhat_raw    ESS
    ____________________    ________    _______    __________    ______
_______    ________    ____

    {'b_Intercept'      }      3.1342    0.17588        2.7889    3.4828
1.0003     20.617     2710
    {'b_x'              }     0.49395    0.25783    -0.0029023    1.0064
0.99967    0.99967     2792
    {'b_kappa_Intercept'}       2.16    0.69821       0.81319    3.6176
1.0007     1.0007     2771
    {'b_kappa_x'        }    -0.20339    0.95816        -2.043
1.6392          1          1     2474
```

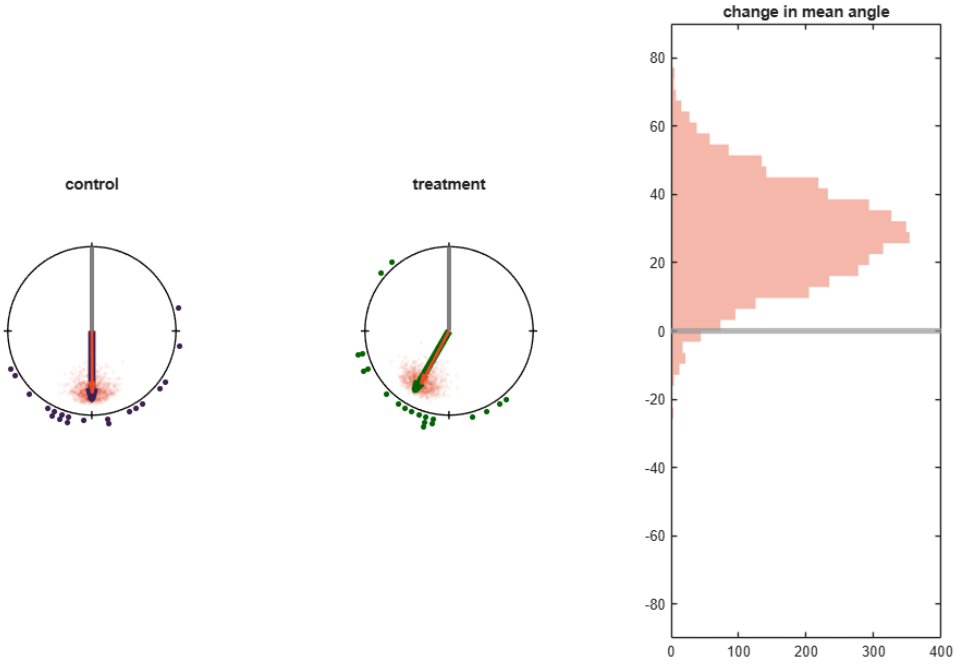
Because the mean direction for the control falls at the boundary of $(-\pi, \pi)$, estimates for the intercept can appear in both positive and negative directions in different chains (`Rhat_raw`). After unwrapping, the chains agree (`Rhat`).

Predictions for the control use `b_Intercept` and `b_kappa_Intercept`, and for the treatment `b_Intercept + b_x` and `b_kappa_Intercept + b_kappa_x`.

```
ctrl = dtDelta.x == 0;
plotTreatment(fitDelta, dtDelta.y(ctrl), dtDelta.y(~ctrl), ...
    deg2rad([180 210]), [3 3], col, 'Change in direction');
bx = getDraw(fitDelta, 'b_x');
nexttile;
vertHist(gca, rad2deg(wrapPi(bx)), 'change in mean angle', [-90 90], col.sd,
0);
saveFig(saveFigures, figDir, 'change_in_direction');
```

```
ciTurnDelta = quantile(rad2deg(unwrapCircular(bx)), [0.025 0.975]);
fprintf('Rightwards turn: %.1f%% of estimates\n', 100*mean(wrapPi(bx) > 0));
fprintf('Median turn: %.0f°; 95%% CI: %.0f° to %.0f°\n', ...
    rad2deg(circMedian(bx)), ciTurnDelta);
```

```
Rightwards turn: 97.5% of estimates
Median turn: 28°; 95% CI: -0° to 58°
```



Uncertainty in the example data results in a wide range of posterior estimates for the treatment direction. Nonetheless, most estimates for change in μ (percentage above) are to the right of 0°, suggesting a rightwards change in direction.

Test the direction change hypothesis

We fit models with treatment effects on κ only, and with no effects of treatment, and compare them with the full model.

```
mKappaDelta = makeModel(dtDelta.y, ...
    'Xmu', ones(nD, 1), 'muNames', {'b_Intercept'}, 'muPrior', [1 0 3*pi/2],
...
    'Xk', [ones(nD, 1) dtDelta.x], 'kNames', {'b_kappa_Intercept',
'b_kappa_x'}, ...
    'kPrior', [3 3; 0 3]);
mNoDelta = makeModel(dtDelta.y, ...
    'Xmu', ones(nD, 1), 'muNames', {'b_Intercept'}, 'muPrior', [1 0 3*pi/2],
...
    'Xk', ones(nD, 1), 'kNames', {'b_kappa_Intercept'}, 'kPrior', [3 3]);
fitKappaDelta = fitUnwrapModel(mKappaDelta, opts, 'model_kappadelta');
fitNoDelta = fitUnwrapModel(mNoDelta, opts, 'model_nodelta');
```

```
printSummary(fitKappaDelta);
printSummary(fitNoDelta);
looDelta = looUnwrap(fitDelta, opts);
looKappaDelta = looUnwrap(fitKappaDelta, opts);
looNoDelta = looUnwrap(fitNoDelta, opts);
looCompare({'model_delta', 'model_kappadelta', 'model_nodelta'}, ...
    {looDelta, looKappaDelta, looNoDelta});
fprintf('ELPD difference (delta - no delta): %s\n', elpdDiff(looDelta,
looNoDelta));
fprintf('ELPD difference (kappa delta - no delta): %s\n',
elpdDiff(looKappaDelta, looNoDelta));
```

```
Fitting model_kappadelta (3 parameters)... 3 s, acceptance ratio [0.93 0.89
0.92 0.93]
Fitting model_nodelta (2 parameters)... 2 s, acceptance ratio [0.94 0.91
0.93 0.93]

model_kappadelta (circular parameters unwrapped, radians)
          Parameter            Mean        SD        Q2_5       Q97_5
Rhat      Rhat_raw    ESS
    ____________________    ________    _______    _______    _______
_______    ________    ____

    {'b_Intercept'      }     -2.9151    0.14508    -3.1983    -2.6317
0.99975     21.667     2937
    {'b_kappa_Intercept'}      2.0115    0.70504    0.69404       3.45
1.0002     1.0002     2856
    {'b_kappa_x'        }    -0.21951    0.99494    -2.1746     1.7646
0.99971    0.99971     2807


model_nodelta (circular parameters unwrapped, radians)
          Parameter            Mean        SD        Q2_5       Q97_5
Rhat      Rhat_raw    ESS
    ____________________    _______    _______    _______    _______
_______    ________    ____

    {'b_Intercept'      }    -2.8986    0.13777    -3.1751    -2.6267
1.0002     26.328     3631
    {'b_kappa_Intercept'}     1.8909    0.47771    0.99907     2.8474
0.99968    0.99968     3522


LOO comparison
          Model            elpd_diff    se_diff     elpd      se_elpd
    ____________________    _________    _______    _______    _______

    {'model_delta'     }           0          0     -52.501    5.2729
    {'model_nodelta'   }    -0.14057      1.829     -52.642    5.6187
    {'model_kappadelta'}     -1.1832     1.7891     -53.684    5.7421

ELPD difference (delta - no delta): 0.1 ± 1.8
ELPD difference (kappa delta - no delta): -1.0 ± 0.3
```

In this case there is uncertainty about the effect of treatment on direction. Although most estimates from the model with a change in direction indicate a rightwards turn (percentage and 95% CI above), these differences could also be explained well by a null model with no change (ELPD difference within its standard error). Since the median estimate is close to the true value of 30°, the model's estimates appear to be robust, if not conclusive. Conversely, there is no evidence for an effect of treatment on concentration, since the model with an effect of treatment on κ only has lower predictive density than the null model, although this difference is small.

Change in concentration



Honeybees performing a "waggle dance" on a horizontal observation hive that view a green light-spot, orient their dances as if viewing the sun. The concentration of a series of dance directions decreases when the brightness of this light spot is reduced.

An experimental treatment may also affect concentration independently of mean direction. In this example the treatment has exactly the same μ (0°) as the control, but κ decreases by $> 80\%$ (from 3 to 0.5). We can use the same model as for the change in direction.

```
dtConc = readtable(fullfile(dataDir, 'conc.csv'));
mConc = setY(mDelta, dtConc.y);
mConc.Xmu(:, 2) = dtConc.x; mConc.Xk(:, 2) = dtConc.x;
fitConc = fitUnwrapModel(mConc, opts, 'model_conc');
printSummary(fitConc);
fprintf('mu estimates to the left of 0°: %.0f%%\n',
100*mean(wrapPi(getDraw(fitConc, 'b_x')) < 0));
```

```
Fitting model_conc (4 parameters)... 4 s, acceptance ratio [0.93 0.91 0.95
0.93]

model_conc (circular parameters unwrapped, radians)
          Parameter             Mean          SD          Q2_5
Q97_5       Rhat       Rhat_raw    ESS
    ____________________    _________    _______    _________
________    _______    _________    ____

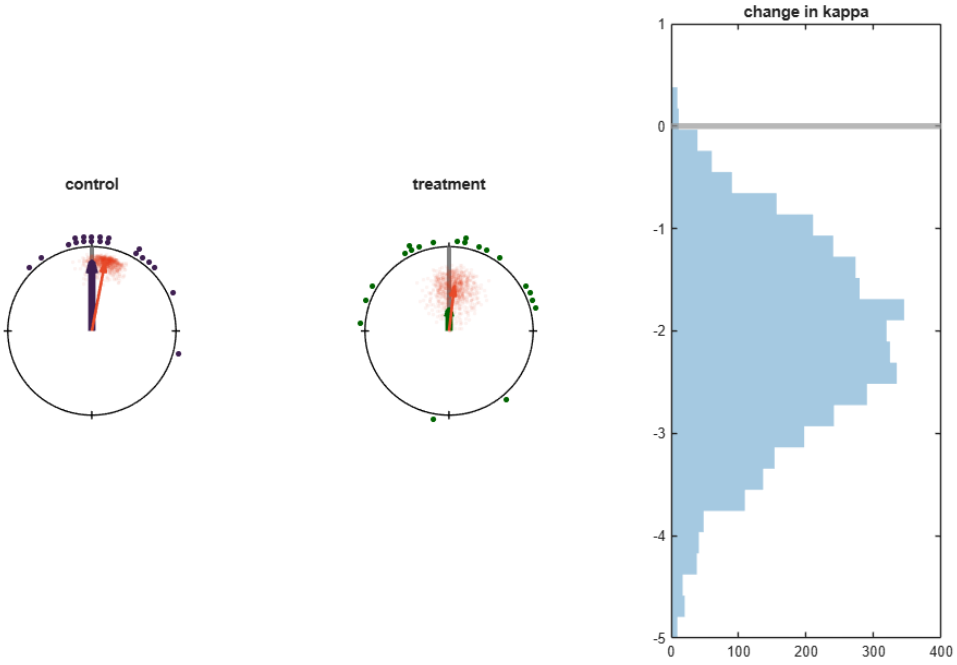
    {'b_Intercept'      }      0.18854    0.13368    -0.076922
0.44941      1.001      1.001      2970
    {'b_x'              }    -0.065185    0.32324      -0.7163
0.59697     1.0001     1.0001      2675
    {'b_kappa_Intercept'}       3.3592    0.91378       1.7242
5.2727     0.99981    0.99981      2684
    {'b_kappa_x'        }      -2.4693     1.1469      -4.8684
-0.46775     1.0014     1.0014      2049

mu estimates to the left of 0°: 59%
```

The model successfully recovers a large change in κ caused by the treatment, alongside only a negligible potential change in μ (similar posterior density to the left and right of 0°). Since κ is estimated on the inverse-softplus scale, estimates for control and treatment are softplus transformed before calculating their difference.

```
ctrl = dtConc.x == 0;
plotTreatment(fitConc, dtConc.y(ctrl), dtConc.y(~ctrl), [0 0], [3 0.5], col,
...
    'Change in concentration');
bkx = getDraw(fitConc, 'b_kappa_x');
bk0 = getDraw(fitConc, 'b_kappa_Intercept');
nexttile;
vertHist(gca, softplus(bk0 + bkx) - softplus(bk0), 'change in kappa', [-5
1], col.kappa, 0);
saveFig(saveFigures, figDir, 'change_in_concentration');
fprintf('Reduction in kappa: %.1f%% of estimates\n', 100*mean(bkx < 0));
```

Reduction in kappa: 99.5% of estimates



Although there is some uncertainty in the particular size of the change in κ, nearly all posterior estimates (percentage above) support a treatment effect that decreases κ.

Test the concentration change hypothesis

```
mConcKappa = setY(mKappaDelta, dtConc.y); mConcKappa.Xk(:, 2) = dtConc.x;
mConcNoKappa = setY(mNoDelta, dtConc.y);
fitConcKappa = fitUnwrapModel(mConcKappa, opts, 'model_conc_kappa');
fitConcNoKappa = fitUnwrapModel(mConcNoKappa, opts, 'model_conc_nokappa');
printSummary(fitConcKappa);
printSummary(fitConcNoKappa);
looConc = looUnwrap(fitConc, opts);
looConcKappa = looUnwrap(fitConcKappa, opts);
looConcNoKappa = looUnwrap(fitConcNoKappa, opts);
looCompare({'model_conc', 'model_conc_kappa', 'model_conc_nokappa'}, ...
```

```
    {looConc, looConcKappa, looConcNoKappa});
fprintf('ELPD difference (kappa only - no kappa): %s\n',
elpdDiff(looConcKappa, looConcNoKappa));
fprintf('ELPD difference (mu & kappa - kappa only): %s\n', elpdDiff(looConc,
looConcKappa));

Fitting model_conc_kappa (3 parameters)... 3 s, acceptance ratio [0.93 0.92
0.93 0.94]
Fitting model_conc_nokappa (2 parameters)... 2 s, acceptance ratio [0.93
0.91 0.93 0.9]

model_conc_kappa (circular parameters unwrapped, radians)
          Parameter           Mean        SD         Q2_5        Q97_5
Rhat      Rhat_raw    ESS
    ____________________    _______    _______    _________    ________
______    ________    ____

    {'b_Intercept'      }    0.18207     0.1208    -0.053798     0.42453
1.0006     1.0006     2869
    {'b_kappa_Intercept'}     3.3968    0.89623       1.7951      5.3119
1.0001     1.0001     2363
    {'b_kappa_x'        }    -2.3776     1.0416       -4.513    -0.47238
0.9999     0.9999     2440


model_conc_nokappa (circular parameters unwrapped, radians)
          Parameter           Mean        SD         Q2_5       Q97_5
Rhat      Rhat_raw    ESS
    ____________________    _______    _______    ________    _______
_______    ________    ____

    {'b_Intercept'      }    0.16122    0.13538    -0.10755    0.43179
0.99985    0.99985     3336
    {'b_kappa_Intercept'}     1.8672    0.46812     0.97268     2.7916
1.0003     1.0003     3374

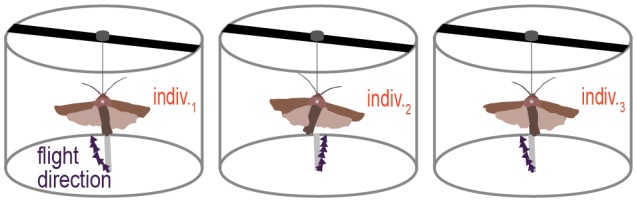

LOO comparison
            Model             elpd_diff    se_diff     elpd      se_elpd
    ____________________    _________    _______    _______    _______

    {'model_conc_kappa'  }           0          0    -50.676     5.8527
    {'model_conc'        }     -1.1164    0.34603    -51.793     5.7399
    {'model_conc_nokappa'}     -2.1058     2.4664    -52.782     6.0858

ELPD difference (kappa only - no kappa): 2.1 ± 2.5
ELPD difference (mu & kappa - kappa only): -1.1 ± 0.3
```

In this case, model comparison is less conclusive than the posterior estimates. The model with an effect of treatment on κ only predicts the data better than the model with no effects of treatment, but this difference is small relative to its standard error. With this sample size, cross-validation therefore provides only weak support for the change in concentration indicated by the posterior estimates. As we might have also expected, the effect of treatment on μ, present in the original model but not the κ-only model, does not improve predictive density, so there is little evidence that the treatment affects μ.

High inter-individual correlation



Tethered Bogong moths can be induced to fly in "flight simulators", which allows repeated measurements of flight directions across multiple individuals simultaneously. Individual directional biases may differ somewhat, but are typically highly correlated, aligning with the population's migratory direction.

In datasets that include repeated measurements across individuals, it is necessary to account for individual differences before attempting to identify any population level trends. In the simplest case, the orientation behaviour of different individuals is very similar: all individuals share a common goal direction β_0, but differ in alignment with that direction by their individual offsets β_i. The distribution across β_i is determined by the hyperparameter κ_μ, the concentration across individual offsets.

In this example, each individual's headings are highly concentrated ($\kappa = 5$) around its own bias $\mu_i = \beta_0 + \beta_i$, and the individual biases are moderately concentrated ($\kappa_\mu = 2$).

The hierarchical model estimates β_0 as a single estimate across individuals and β_i as an offset for each individual, with the prior $\beta_i \sim vM(0, \kappa_\mu)$ and, as expected for high correlation, $\log \kappa_\mu \sim N(\log 15, 0.6)$. Individual differences in κ follow a normal distribution (inverse-softplus scale) with standard deviation $\sigma_\kappa \sim HalfT(3, 0, 0.5)$, using a non-centred parameterisation. Circular parameters are summarised after unwrapping: for some individuals, estimates from different chains are separated around the circle (`Rhat_raw`), which can make it appear that the model has not converged.

```
dtHigh = readData(dataDir, 'highcorr.csv');
trueHigh = readtable(fullfile(dataDir, 'highcorr_true.csv'));
mHigh = makeIndividualModel(dtHigh.y, dtHigh.ID, ...
    'beta0Prior', [1 0 3*pi/2], 'kappaPrior', [3 3], 'sdScale', 0.5);
fitHigh = fitUnwrapModel(mHigh, opts, 'model_highcorr');
printSummary(fitHigh);
```

```
Fitting model_highcorr (24 parameters)... 81 s, acceptance ratio [0.95 0.94
0.94 0.93]

model_highcorr (circular parameters unwrapped, radians)
              Parameter                  Mean         SD          Q2_5
Q97_5       Rhat       Rhat_raw    ESS
    ___________________________________    __________    _______    _________
_________    _______    _________    ____

    {'b_beta0_Intercept'         }     0.053399    0.24958    -0.45139
0.52891      1.001      1.001       371
    {'b_kappa_Intercept'         }       5.0377    0.52739      4.0374
6.134     0.99985    0.99985     2716
    {'b_betai_ID1'               }     -0.41956    0.26625    -0.94166
0.094288     1.0006     1.0006      395
```

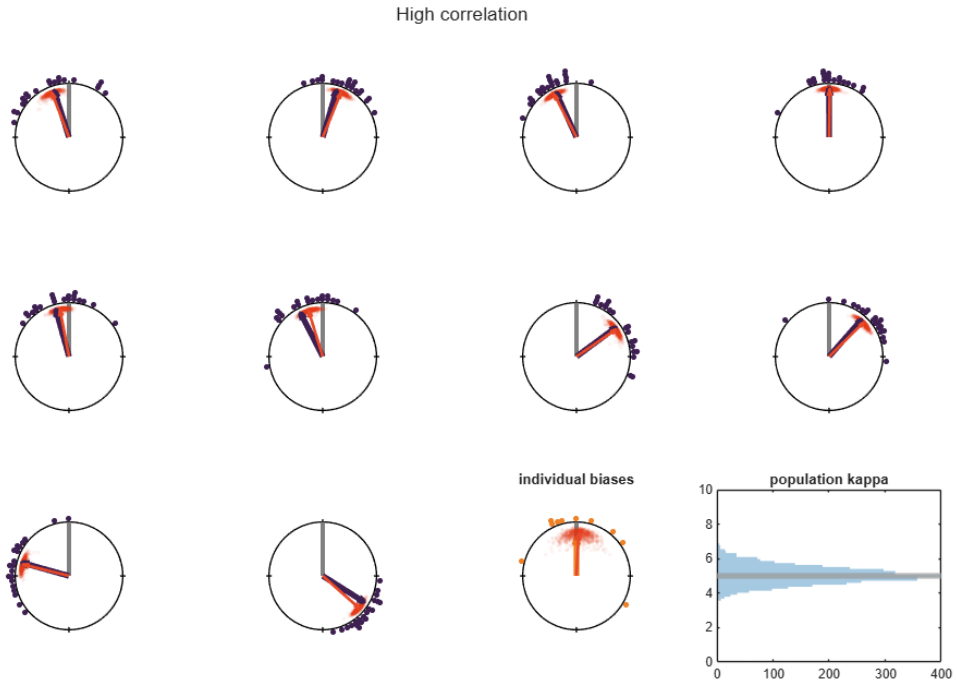
```
    {'b_betai_ID2'               }      0.33387    0.26266    -0.17198
0.84712     1.0013     1.0013      477
    {'b_betai_ID3'               }     -0.53333    0.26209     -1.0371
-0.017454     1.0005     1.0005      486
    {'b_betai_ID4'               }    -0.041884    0.26265    -0.55305
0.46311     1.0005     1.0005      496
    {'b_betai_ID5'               }     -0.24511    0.26389    -0.74905
0.27872     1.0018     1.0018      512
    {'b_betai_ID6'               }     -0.35298    0.26687    -0.87351
0.17635     1.0006     1.0006      462
    {'b_betai_ID7'               }      0.91577    0.26821     0.41174
1.4591     1.0015     1.0015      515
    {'b_betai_ID8'               }      0.73659    0.26714     0.23589
1.2711     1.0009     1.0009      495
    {'b_betai_ID9'               }       -1.379     0.2726     -1.9208
-0.83816     1.0003     1.0003      512
    {'b_betai_ID10'              }       2.2428    0.27195      1.7197
2.7636     1.0005     1.0005      523
    {'softkappamu'               }       2.7364    0.83901      1.2309
4.5687     1.0009     1.0009     2177
    {'kappa_mu'                  }       2.8192    0.78394      1.4871
4.579     1.0009     1.0009     2127
    {'sd_ID__kappa_Intercept'    }      0.54557    0.46418    0.016155
1.7466     1.0005     1.0005     1713
    {'r_ID__kappa[1,Intercept]' }     -0.39096    0.67621     -2.2306
0.48697    0.99959    0.99959     2149
    {'r_ID__kappa[2,Intercept]' }      0.26406    0.67044    -0.76221
2.0623     1.0015     1.0015     2238
    {'r_ID__kappa[3,Intercept]' }      0.33711    0.73054    -0.64401
2.3811     1.0013     1.0013     1945
    {'r_ID__kappa[4,Intercept]' }      0.24847    0.68219    -0.86102
2.1049    0.99985    0.99985     2621
    {'r_ID__kappa[5,Intercept]' }    0.0095491    0.58258     -1.2378
1.3199    0.99994    0.99994     2513
    {'r_ID__kappa[6,Intercept]' }     -0.12949     0.5814     -1.5667
0.95936          1          1     2935
    {'r_ID__kappa[7,Intercept]' }     -0.19225    0.59584      -1.668
0.91121    0.99976    0.99976     2245
    {'r_ID__kappa[8,Intercept]' }     -0.12583    0.58292     -1.5433
0.96357    0.99985    0.99985     2457
    {'r_ID__kappa[9,Intercept]' }     -0.23357     0.6323     -1.8466
0.7914    0.99971    0.99971     2661
    {'r_ID__kappa[10,Intercept]'}      0.34615    0.74819    -0.65531
2.364     1.0012     1.0012     2375
```

The plots show each individual's data (dark purple points) with its true mean vector (dark purple arrow), the posterior median mean vector (red arrow) and a cloud of posterior mean vectors, followed by the individual biases (orange points) and the population concentration κ.

```
plotIndividuals(fitHigh, dtHigh, deg2rad(trueHigh.mu),
5.0*ones(height(trueHigh), 1), ...
    0, 2.0, 5.0, col, 'High correlation');
saveFig(saveFigures, figDir, 'high_correlation');
```

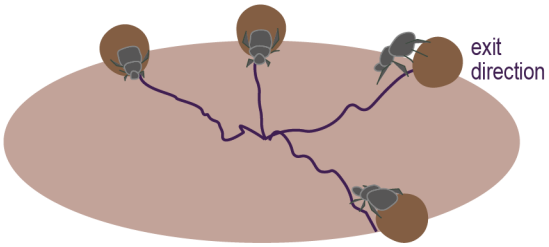
```
fprintf('softplus(b_kappa_Intercept) > true kappa: %.1f%%\n', ...
    100*mean(softplus(getDraw(fitHigh, 'b_kappa_Intercept')) > 5.0));
```

```
softplus(b_kappa_Intercept) > true kappa: 52.2%
```



The estimates of κ_μ have a slight positive bias, due to a combination of low sample size and our prior to higher values. The model nonetheless recovers good estimates of β_0 and β_i, as well as population κ, the concentration of the average individual, which needs to be distinguished from individual differences and the concentration across individuals.

Low inter-individual correlation



When rolling a dung ball, ball-rolling dung beetles can be repeatedly returned to the centre of a circular arena on exiting, resulting in repeated measurements of exit direction. While a single individual exits consistently in the same direction, there is typically no relationship between individual directional biases.

In this example each individual's headings have a high concentration ($\kappa = 5$), but the concentration across individual directional biases is low ($\kappa_\mu = 0.1$). The same model and priors are used.

```
dtLow = readData(dataDir, 'lowcorr.csv');
trueLow = readtable(fullfile(dataDir, 'lowcorr_true.csv'));
```

```
mLow = makeIndividualModel(dtLow.y, dtLow.ID, ...
    'beta0Prior', [1 0 3*pi/2], 'kappaPrior', [3 3], 'sdScale', 0.5);
fitLow = fitUnwrapModel(mLow, opts, 'model_lowcorr');
printSummary(fitLow);
plotIndividuals(fitLow, dtLow, deg2rad(trueLow.mu),
5.0*ones(height(trueLow), 1), ...
    0, 0.1, 5.0, col, 'Low correlation');
saveFig(saveFigures, figDir, 'low_correlation');
ciBeta0Low = quantile(rad2deg(unwrapCircular(getDraw(fitLow,
'b_beta0_Intercept'))), [0.025 0.5 0.975]);
fprintf('beta0 (degrees clockwise) 2.5%%, 50%%, 97.5%%: %s; 95%% CI width:
%.0f°\n', ...
    mat2str(round(ciBeta0Low)), ciBeta0Low(3) - ciBeta0Low(1));
fprintf('softplus(b_kappa_Intercept) > true kappa: %.1f%%\n', ...
    100*mean(softplus(getDraw(fitLow, 'b_kappa_Intercept')) > 5.0));
```

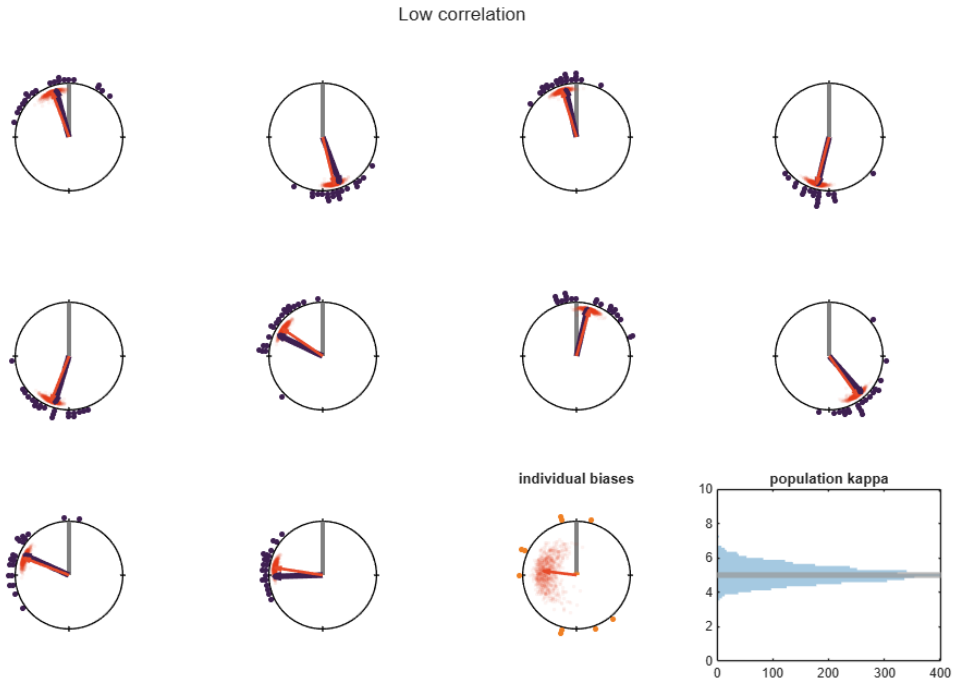
```
Fitting model_lowcorr (24 parameters)... 152 s, acceptance ratio [0.94 0.93
0.93 0.93]

model_lowcorr (circular parameters unwrapped, radians)
              Parameter                 Mean          SD          Q2_5
Q97_5        Rhat       Rhat_raw     ESS
    ___________________________________    ___________    _______    ________
________    _______    _________    ____

    {'b_beta0_Intercept'         }       -1.4474     0.48362     -2.3763
-0.43551     1.0052     6.5936       455
    {'b_kappa_Intercept'         }        5.0404     0.54171      4.0521
6.2033     0.99972    0.99972      2421
    {'b_betai_ID1'               }        1.0843     0.49299    0.052658
2.0478     1.0051     6.4738       472
    {'b_betai_ID2'               }       -1.9349     0.49852     -2.9687
-0.96643     1.0053     7.4579       469
    {'b_betai_ID3'               }        1.1453     0.49012      0.1252
2.1055      1.005     7.5776       462
    {'b_betai_ID4'               }       -1.4336     0.49463     -2.4762
-0.49929     1.0051     6.4203       472
    {'b_betai_ID5'               }        -1.315     0.49382     -2.3522
-0.3482     1.0047     6.4297       475
    {'b_betai_ID6'               }       0.47267     0.48783      -0.544
1.4295     1.0047     10.588       469
    {'b_betai_ID7'               }        1.7191     0.49695     0.68629
2.6838     1.0046     12.699       474
    {'b_betai_ID8'               }       -2.3126     0.50235     -3.3779
-1.3408     1.0047     1.0047       464
    {'b_betai_ID9'               }       0.24018     0.48838    -0.76357
1.183     1.0056     6.5324       474
    {'b_betai_ID10'              }      0.028066     0.48603    -0.98823
0.95981     1.0048     10.607       477
    {'softkappamu'               }        1.3663      0.5847      0.2611
2.5796          1          1      2313
    {'kappa_mu'                  }        1.6205     0.46438     0.83219
2.6527          1          1      2212
    {'sd_ID__kappa_Intercept'    }       0.55323     0.44973    0.024481
```

```
1.6536      1.001      1.001     1156
    {'r_ID__kappa[1,Intercept]' }      -0.40028     0.68602     -2.2186
0.55364     1.0009     1.0009     1413
    {'r_ID__kappa[2,Intercept]' }       0.23962     0.65495    -0.83364
1.9097    0.99963    0.99963     2139
    {'r_ID__kappa[3,Intercept]' }       0.32229     0.66788       -0.61
2.0926     1.0015     1.0015     1530
    {'r_ID__kappa[4,Intercept]' }       0.25771     0.65359    -0.72167
1.9521     1.0002     1.0002     2458
    {'r_ID__kappa[5,Intercept]' }    0.00015159      0.5954     -1.3364
1.2719     1.0001     1.0001     2515
    {'r_ID__kappa[6,Intercept]' }      -0.12336     0.59973     -1.5539
1.0292     1.0017     1.0017     2431
    {'r_ID__kappa[7,Intercept]' }      -0.18316     0.60193     -1.7078
0.84204     1.0002     1.0002     2490
    {'r_ID__kappa[8,Intercept]' }      -0.11758     0.60093     -1.6197
0.97149      1.001      1.001     2284
    {'r_ID__kappa[9,Intercept]' }      -0.22901     0.61209     -1.8209
0.73093     1.0012     1.0012     2130
    {'r_ID__kappa[10,Intercept]'}       0.34783     0.73585    -0.62352
2.2794     1.0029     1.0029     1842

beta0 (degrees clockwise) 2.5%, 50%, 97.5%: [-136 -84 -25]; 95% CI width:
111°
softplus(b_kappa_Intercept) > true kappa: 51.2%
```



As for the high-correlation example, the model overestimates κ_μ, but recovers accurate estimates of β_i and population κ. Perhaps unsurprisingly given the set of individual heading biases, the population mean direction β_0 was not

successfully recovered, but the broad credible interval around the estimate suggests considerable uncertainty about the true direction. This may lead us to ask whether or not there is a shared population mean direction.

Test shared heading bias hypothesis

We fit a model without κ_μ: β_0 is anchored at 0° and the individual biases have a (near) uniform prior, $\beta_i \sim vM(0, \approx 0)$. A higher target acceptance ratio gives smaller sampling steps: slower, but more robust sampling. Without the information from κ_μ the posterior for each β_i is periodic and unbounded, so step size tuning may warn that it is attempting a large number of leapfrog steps.

```
mNoCorr = makeIndividualModel(dtLow.y, dtLow.ID, ...
    'beta0Prior', [], 'kappaPrior', [3 3], 'sdScale', 0.5, 'betaiKappa',
1e-16);
optsSlow = opts; optsSlow.targetAccept = 0.97;
fitNoCorr = fitUnwrapModel(mNoCorr, optsSlow, 'model_nocorr');
printSummary(fitNoCorr);
looLow = looUnwrap(fitLow, opts);
looNoCorr = looUnwrap(fitNoCorr, optsSlow);
looCompare({'model_lowcorr', 'model_nocorr'}, {looLow, looNoCorr});
fprintf('ELPD difference (kappa_mu - no kappa_mu): %s\n', elpdDiff(looLow,
looNoCorr));
```

```
Fitting model_nocorr (22 parameters)... Warning: Stepsize tuning via dual-
averaging is attempting a large number of
leapfrog steps (2564). To speed up tuning and sampling, consider decreasing
the
simulation length by decreasing StepSize or NumSteps.
121 s, acceptance ratio [0.99 0.99 0.99 1]

model_nocorr (circular parameters unwrapped, radians)
              Parameter                Mean          SD          Q2_5
Q97_5       Rhat       Rhat_raw    ESS
    ___________________________________    _________    _________    _________
_________    _______    _________    ____

    {'b_kappa_Intercept'        }      5.0438      0.52344       4.0608
6.1416    0.99987    0.99987     2753
    {'b_betai_ID1'              }    -0.33858      0.11604    -0.57277
-0.10412     0.9999     0.9999     3151
    {'b_betai_ID2'              }      2.8935      0.10199       2.7014
3.091     0.9999     0.9999     3855
    {'b_betai_ID3'              }    -0.28599      0.10277    -0.48688
-0.077574    0.99981    0.99981     3357
    {'b_betai_ID4'              }     -2.8955      0.10341     -3.1019
-2.6927     0.9997     30.397     3310
    {'b_betai_ID5'              }      -2.776      0.10706     -2.9785
-2.5622     1.0001     29.354     3356
    {'b_betai_ID6'              }    -0.96967       0.1053     -1.1729
-0.76192     1.0012     1.0012     3719
    {'b_betai_ID7'              }     0.28681      0.10958     0.064191
0.50094    0.99989    0.99989     3492
    {'b_betai_ID8'              }       2.506      0.10791       2.291
2.7212    0.99959     29.112     3993
```

```
    {'b_betai_ID9'                }     -1.2027     0.11156     -1.4211
-0.98595    0.99968     32.529     3605
    {'b_betai_ID10'               }     -1.4158    0.099538     -1.6179
-1.215          1           1     3658
    {'sd_ID__kappa_Intercept'     }     0.56009     0.45022    0.022867
1.6424     1.0022     1.0022     1573
    {'r_ID__kappa[1,Intercept]' }    -0.41487      0.6945     -2.2193
0.50525     1.0009     1.0009     2048
    {'r_ID__kappa[2,Intercept]' }     0.22854      0.6475    -0.84329
1.8566     1.0006     1.0006     2968
    {'r_ID__kappa[3,Intercept]' }     0.33681     0.70292    -0.60434
2.1327     1.0005     1.0005     2697
    {'r_ID__kappa[4,Intercept]' }     0.25681     0.66635    -0.78415
2.0032     1.0005     1.0005     3124
    {'r_ID__kappa[5,Intercept]' }    -0.00597     0.59485     -1.3023
1.2541     1.0015     1.0015     2771
    {'r_ID__kappa[6,Intercept]' }    -0.12338     0.58514     -1.5661
0.99659    0.99997    0.99997     2475
    {'r_ID__kappa[7,Intercept]' }    -0.19373     0.60892      -1.699
0.85787     1.0014     1.0014     2652
    {'r_ID__kappa[8,Intercept]' }    -0.12905     0.56874     -1.4995
0.98064    0.99998    0.99998     2730
    {'r_ID__kappa[9,Intercept]' }     -0.2247     0.61203     -1.7616
0.82438    0.99977    0.99977     2589
    {'r_ID__kappa[10,Intercept]'}     0.33707     0.71977    -0.66822
2.1971     1.0015     1.0015     2724

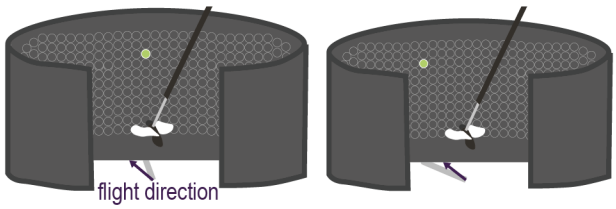

LOO comparison
          Model           elpd_diff    se_diff     elpd      se_elpd
    ________________    _________    _______    _______    _______

    {'model_nocorr' }           0           0    -141.58    11.423
    {'model_lowcorr'}    -0.21778     0.45392     -141.8    11.533

ELPD difference (kappa_mu - no kappa_mu): -0.2 ± 0.5
```

Although this dataset was simulated with a von Mises distribution with very low concentration ($\kappa_\mu = 0.1$), the distribution of individual headings does not detectably differ from uniform. This is plausible for animals without a shared goal direction, and at this level of concentration a considerably larger number of individuals would be needed to recover any group-level bias. Nonetheless, both models recover the concentration of the average individual's headings ($\kappa \approx 5$).

Variable individual parameters



Tethered vinegar flies (Drosophila) can be induced to fly in a "virtual reality" LED arena, in which their wing motion causes visual cues to rotate. A green light spot induces orientation behaviour, with a weak population level bias towards the light, but highly variable performance across individuals.

In this example, there is a moderate concentration across individual biases ($\kappa_\mu = 1$), and the average individual's headings are somewhat concentrated ($\alpha_0 = 1.5$), but the concentration within each individual's headings varies ($\sigma_\kappa = 0.5$), including a $\approx 0.1\%$ chance of completely uniform heading distributions ($\kappa = 0$). Despite the modest inter-individual correlation, 4 individuals diverge from the population mean direction by more than 90°, and individual concentrations vary from $\kappa = 1.14$ to $\kappa = 2.66$.

We use an unbiased (near uniform) von Mises prior for β_0, a prior expecting lower average concentration, $\alpha_0 \sim N(1, 3)$, and a wider prior for individual differences in concentration, $\sigma_\kappa \sim HalfT(3, 0, 1)$.

```
dtVar = readData(dataDir, 'var.csv');
trueVar = readtable(fullfile(dataDir, 'var_true.csv'));
mVar = makeIndividualModel(dtVar.y, dtVar.ID, ...
    'beta0Prior', [2 0 1e-16], 'kappaPrior', [1 3], 'sdScale', 1.0);
fitVar = fitUnwrapModel(mVar, opts, 'model_var');
printSummary(fitVar);
plotIndividuals(fitVar, dtVar, deg2rad(trueVar.mu), trueVar.kappa, ...
    0, 1.0, 1.5, col, 'Variable individuals');
saveFig(saveFigures, figDir, 'variable_individuals');
fprintf('beta0 > 0°: %.1f%% of estimates\n', ...
    100*mean(unwrapCircular(getDraw(fitVar, 'b_beta0_Intercept')) > 0));
```

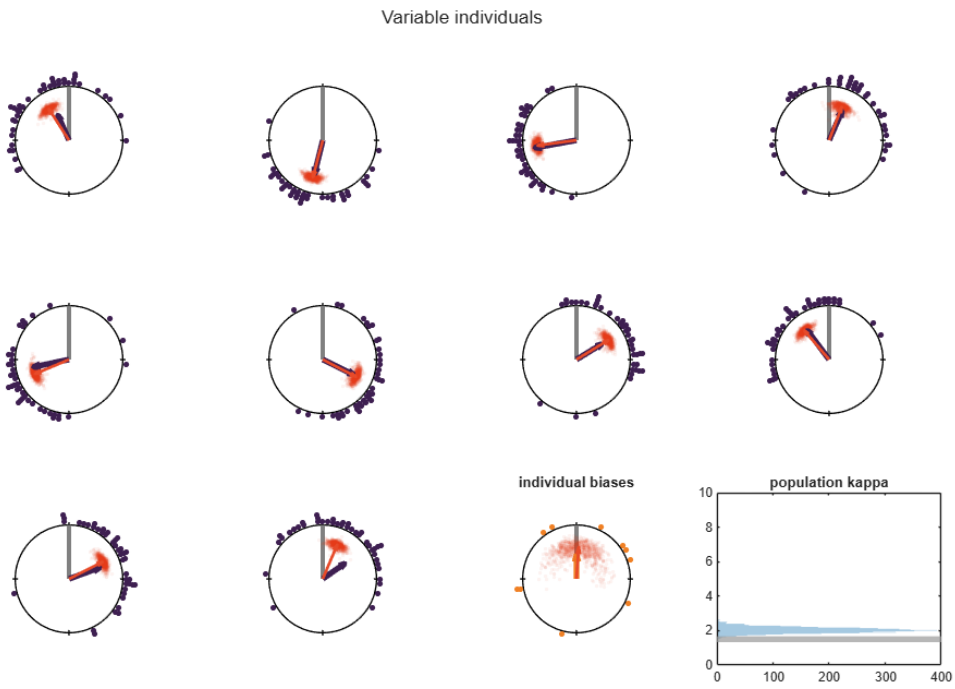
```
Fitting model_var (24 parameters)... 137 s, acceptance ratio [0.94 0.93 0.93
0.91]

model_var (circular parameters unwrapped, radians)
              Parameter                  Mean          SD          Q2_5
Q97_5       Rhat      Rhat_raw    ESS
    ___________________________________    _________    _______    _________
_________    _______    _________    ____

    {'b_beta0_Intercept'         }     0.078539    0.60283     -1.0656
1.375     1.0043     1.0043      419
    {'b_kappa_Intercept'         }       1.8791    0.19331      1.5113
2.2766    0.99981    0.99981     2161
    {'b_betai_ID1'               }     -0.65368     0.6093     -1.9609
0.49529     1.0036     1.0036      436
    {'b_betai_ID2'               }      -2.9606    0.62547     -4.2902
-1.7962     1.0047      8.234      425
    {'b_betai_ID3'               }      -1.7847    0.61779     -3.1227
-0.61876     1.0044     5.9315      432
    {'b_betai_ID4'               }      0.28128    0.60962    -0.99759
1.4344     1.0039     5.2474      437
    {'b_betai_ID5'               }      -2.0475    0.62885     -3.4058
-0.87309     1.0033     5.0345      436
    {'b_betai_ID6'               }        1.967     0.6226     0.61686
3.1312     1.0035     1.0035      433
    {'b_betai_ID7'               }      0.94415    0.60743    -0.33848
2.1014     1.0042     1.0042      428
    {'b_betai_ID8'               }     -0.78757    0.60861     -2.0974
```

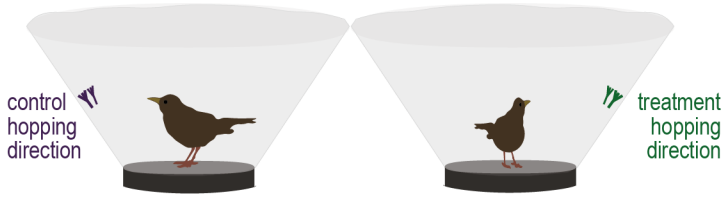
```
0.35032     1.0043     1.0043      433
    {'b_betai_ID9'               }       1.0247    0.60759    -0.22922
2.1673     1.0046     1.0046      426
    {'b_betai_ID10'              }      0.32669    0.60977    -0.98045
1.4743     1.0037     1.0037      424
    {'softkappamu'               }       1.2335    0.54487      0.2397
2.3143     1.0001     1.0001     2825
    {'kappa_mu'                  }       1.5145    0.42119     0.82016
2.4086     1.0002     1.0002     2752
    {'sd_ID__kappa_Intercept'    }      0.32327    0.22146    0.018194
0.85017    0.99979    0.99979      922
    {'r_ID__kappa[1,Intercept]' }    -0.052597    0.27741    -0.72311
0.48101    0.99963    0.99963     2246
    {'r_ID__kappa[2,Intercept]' }      0.16069    0.30639    -0.34231
0.88805     1.0011     1.0011     2061
    {'r_ID__kappa[3,Intercept]' }      0.33326    0.40026    -0.17536
1.3503    0.99987    0.99987     1319
    {'r_ID__kappa[4,Intercept]' }     -0.27335    0.32967     -1.0539
0.19333     1.0002     1.0002     1399
    {'r_ID__kappa[5,Intercept]' }     -0.10321    0.27815    -0.76752
0.37182    0.99962    0.99962     2390
    {'r_ID__kappa[6,Intercept]' }    -0.033703     0.2782    -0.65375
0.52988     1.0005     1.0005     2442
    {'r_ID__kappa[7,Intercept]' }    -0.035674    0.27369    -0.64744
0.50984     1.0003     1.0003     2730
    {'r_ID__kappa[8,Intercept]' }     0.078176    0.27194     -0.4725
0.69525     1.0017     1.0017     2519
    {'r_ID__kappa[9,Intercept]' }     0.039937    0.27992    -0.52094
0.64987     1.0009     1.0009     2825
    {'r_ID__kappa[10,Intercept]'}    -0.082041    0.28045    -0.71096
0.455    0.99994    0.99994     2183

beta0 > 0°: 52.6% of estimates
```



The model accounts for individual differences in both directional bias (β_i) and concentration (α_i), and the hyper-parameters (κ_μ and σ_κ) are recovered quite accurately. Individuals with very low concentration and large deviations from the population mean are estimated to be slightly closer to the population mean than their true values (shrinkage). The population mean direction is recovered, with a wide posterior distribution for β_0 centred close to the true value.

Individual heading changes



Birds exhibiting migratory restlessness hop repeatedly in their intended flight direction, allowing this to be recorded repeatedly for the same individuals across multiple nights. Individuals may differ somewhat in their hopping directions, but are generally aligned provided they detect the same compass cues. A treatment that affects their perception of these cues causes the entire population to change their hopping direction.

In this example, the treatment induces a population-level rightwards turn of 30°. The average individual's headings are concentrated ($\kappa = 3$) but vary greatly between individuals ($\sigma_\kappa = 2.5$), and heading biases are highly concentrated across individuals ($\kappa_\mu = 4$). There are large differences between individual responses: one individual turns to the left after treatment, and individual concentrations range from $\kappa = 1.4$ to $\kappa = 8.8$.

The treatment x predicts β_0 (`beta0 ~ 1 + x`), β_i (`betai ~ 0 + ID + x:ID`) and κ (kappa ~ 0 + Intercept + x, with individual effects on both the intercept and x). Individual biases in the control follow $\beta_i \sim vM(0, \kappa_\mu)$,

and in the treatment $\beta_i + \beta_{i,x} \sim vM(0, \kappa_{\mu,x})$, where $\kappa_{\mu,x} = \log(\exp(k_\mu + k_{\mu,x}) + 1)$ and $k_{\mu,x} \sim N(0,1)$. Models with effects of treatment on κ only ('kappa model') and with no effects of treatment ('null model') are fitted for comparison, all with a high target acceptance ratio.

```
dtHd = readData(dataDir, 'hd.csv');
trueHd = readtable(fullfile(dataDir, 'hd_true.csv'));
mHd = makeIndividualModel(dtHd.y, dtHd.ID, ...
    'x', dtHd.x, 'muTreatment', true, 'kappaTreatment', true, ...
    'beta0Prior', [1 0 3*pi/2; 1 0 pi/2], 'kappaPrior', [3 3; 0 3], ...
    'sdScale', [3 3]);
mHdKappa = makeIndividualModel(dtHd.y, dtHd.ID, ...
    'x', dtHd.x, 'muTreatment', false, 'kappaTreatment', true, ...
    'beta0Prior', [1 0 3*pi/2], 'kappaPrior', [3 3; 0 3], 'sdScale', [3 3]);
mHdNull = makeIndividualModel(dtHd.y, dtHd.ID, ...
    'beta0Prior', [1 0 3*pi/2], 'kappaPrior', [3 3], 'sdScale', 3);
fitHdTreat = fitUnwrapModel(mHd, optsSlow, 'model_hd_treatment');
fitHdKappa = fitUnwrapModel(mHdKappa, optsSlow, 'model_hd_kappa');
fitHdNull = fitUnwrapModel(mHdNull, optsSlow, 'model_hd_null');
printSummary(fitHdTreat);
printSummary(fitHdKappa);
printSummary(fitHdNull);
for v = {'b_kappa_x', 'cor_ID__kappa_Intercept__kappa_x'}
    fprintf('%s 95%% CI: %.2f to %.2f\n', v{1}, quantile(getDraw(fitHdTreat,
v{1}), [0.025 0.975]));
end
```

```
Fitting model_hd_treatment (29 parameters)... Warning: Stepsize tuning via
dual-averaging is attempting a large number of
leapfrog steps (2996). To speed up tuning and sampling, consider decreasing
the
simulation length by decreasing StepSize or NumSteps.
216 s, acceptance ratio [0.99 0.98 0.98 0.98]
Fitting model_hd_kappa (22 parameters)... 151 s, acceptance ratio [0.97 0.99
0.98 0.98]
Fitting model_hd_null (14 parameters)... 56 s, acceptance ratio [0.99 0.99
0.99 0.98]

model_hd_treatment (circular parameters unwrapped, radians)
                Parameter                       Mean         SD
Q2_5       Q97_5       Rhat      Rhat_raw    ESS
    ____________________________________    ___________    _______
_________    ________    _______    _________    ____

    {'b_beta0_Intercept'              }      -0.15573     0.2134
-0.59641     0.27184     1.0009     1.0009      846
    {'b_beta0_x'                      }       0.85906    0.33287
0.17294      1.5337     1.0005     1.0005      828
    {'b_kappa_Intercept'              }        3.2162     1.1486
1.02       5.6967     1.0007     1.0007     1228
    {'b_kappa_x'                      }        1.7874     1.2522
-0.49366      4.5189    0.99963    0.99963     1904
    {'b_betai_ID1'                    }     -0.092757    0.27967
-0.65703     0.47556     1.0005     1.0005     1455
    {'b_betai_ID2'                    }       0.24883    0.23574
```

```
-0.23359     0.71845     1.0011     1.0011      946
    {'b_betai_ID3'                      }      -0.20115    0.22558
-0.63983     0.24049     1.0006     1.0006      902
    {'b_betai_ID4'                      }       0.11031    0.25115
-0.38791      0.6304      1.002      1.002     1082
    {'b_betai_ID5'                      }      -0.10809    0.23133
-0.55703     0.36265     1.0007     13.586      936
    {'b_betai_ID1:x'                    }      -0.45655    0.40693
-1.2587     0.34212     1.0003     1.0003     1294
    {'b_betai_ID2:x'                    }       0.24924     0.3589
-0.46142     0.97583     1.0001     1.0001      912
    {'b_betai_ID3:x'                    }       0.56952    0.34788
-0.13      1.2696     1.0007     1.0007      879
    {'b_betai_ID4:x'                    }       -1.2975    0.39151
-2.0896    -0.56545     1.0016     1.0016     1029
    {'b_betai_ID5:x'                    }       0.86006    0.35081
0.15674      1.5854     1.0001     1.0001      880
    {'softkappamu'                      }        5.6437     1.8321
2.7264      9.8262     1.0012     1.0012     1989
    {'softkappamux'                     }      -0.71809    0.93689
-2.59      1.1248     0.9998     0.9998     3249
    {'kappa_mu'                         }        5.6551     1.8195
2.7898      9.8263     1.0012     1.0012     1981
    {'kappa_mu_x'                       }        4.9508     1.8896
1.9772      9.3988     1.0005     1.0005     1783
    {'sd_ID__kappa_Intercept'           }        2.4104     1.1717
0.81489      5.2544      1.001      1.001     1748
    {'sd_ID__kappa_x'                   }        2.2212     1.5373
0.1523      6.0252     1.0005     1.0005     1492
    {'cor_ID__kappa_Intercept__kappa_x'}       0.50066    0.45475
-0.68366     0.98807     1.0006     1.0006     1954
    {'r_ID__kappa[1,Intercept]'         }       -1.9866      1.248
-4.6727     0.23489    0.99993    0.99993     1303
    {'r_ID__kappa[2,Intercept]'         }       0.10262     1.2392
-2.4833      2.5919     1.0002     1.0002     1496
    {'r_ID__kappa[3,Intercept]'         }        2.9454     1.6984
0.10451      6.7848     1.0008     1.0008     1919
    {'r_ID__kappa[4,Intercept]'         }       -1.3254     1.2194
-3.9784      0.8876     1.0002     1.0002     1219
    {'r_ID__kappa[5,Intercept]'         }        1.0549     1.3551
-1.5815      3.9371     1.0013     1.0013     1657
    {'r_ID__kappa[1,x]'                 }        -1.359     1.3893
-4.4658     0.89393     1.0002     1.0002     1730
    {'r_ID__kappa[2,x]'                 }    -0.0068092     1.3803
-2.878      3.1132          1          1     1907
    {'r_ID__kappa[3,x]'                 }         2.606     2.6266
-0.78856      9.1922     1.0011     1.0011     1693
    {'r_ID__kappa[4,x]'                 }       -1.0072     1.3498
-4.0434      1.2538    0.99964    0.99964     1865
    {'r_ID__kappa[5,x]'                 }        1.4058     1.8108
-1.3086      5.9122    0.99999    0.99999     1857


model_hd_kappa (circular parameters unwrapped, radians)
```

Parameter	Mean	SD	Q2_5	Q97_5	Rhat	Rhat_raw	ESS
{'b_beta0_Intercept'}	0.63558	0.25687	0.13995	1.1482	1.0003	1.0003	875
{'b_kappa_Intercept'}	0.70336	0.85056	-0.91913	2.5037	0.9997	0.9997	1219
{'b_kappa_x'}	2.9568	2.0815	-1.5454	6.7176	1.0003	1.0003	1677
{'b_betai_ID1'}	-0.64659	0.28677	-1.2085	-0.076585	1.0007	1.0007	1050
{'b_betai_ID2'}	0.30793	0.2847	-0.26425	0.88215	1.0005	1.0005	1044
{'b_betai_ID3'}	0.40153	0.26332	-0.12096	0.91753	1.0004	1.0004	910
{'b_betai_ID4'}	-0.94116	0.284	-1.5049	-0.41002	1.0002	1.0002	985
{'b_betai_ID5'}	0.7942	0.26417	0.27577	1.3141	1.0003	1.0003	925
{'softkappamu'}	5.0411	1.9124	2.1901	9.5213	1.002	1.002	1811
{'kappa_mu'}	5.0617	1.891	2.2962	9.5214	1.0021	1.0021	1797
{'sd_ID__kappa_Intercept'}	1.8668	0.96681	0.56955	4.2834	1.0003	1.0003	1593
{'sd_ID__kappa_x'}	5.7698	2.3498	2.4799	11.508	0.99991	0.99991	1465
{'cor_ID__kappa_Intercept__kappa_x'}	-0.73181	0.27446	-0.99455	-0.0043201	1.0015	1.0015	1579
{'r_ID__kappa[1,Intercept]'}	0.37717	0.92707	-1.6486	2.197	0.99958	0.99958	1415
{'r_ID__kappa[2,Intercept]'}	0.2098	1.0317	-1.9363	2.31	0.99988	0.99988	1613
{'r_ID__kappa[3,Intercept]'}	-2.2897	1.7716	-6.9084	0.11765	1.0002	1.0002	2003
{'r_ID__kappa[4,Intercept]'}	0.78714	0.93832	-1.0416	2.6962	1.0011	1.0011	1588
{'r_ID__kappa[5,Intercept]'}	-2.6218	1.5863	-6.531	-0.35433	1.0003	1.0003	1843
{'r_ID__kappa[1,x]'}	-2.3402	2.1944	-6.4918	2.3943	1.0006	1.0006	1885
{'r_ID__kappa[2,x]'}	-0.0021861	2.6087	-5.0111	5.3408	1.0003	1.0003	1974
{'r_ID__kappa[3,x]'}	9.2721	4.6632	1.8144	19.99	1.0004	1.0004	1925
{'r_ID__kappa[4,x]'}	-2.1128	2.237	-6.3812	2.5855	1.0005	1.0005	1954
{'r_ID__kappa[5,x]'}	7.6858	3.7527	1.6616	16.335	1.0005	1.0005	1795

model_hd_null (circular parameters unwrapped, radians)

```
             Parameter                  Mean        SD         Q2_5
Q97_5       Rhat      Rhat_raw    ESS
    ___________________________________    _________    _______    _________
_________    _______    _________    ____

    {'b_beta0_Intercept'       }     0.26308    0.16812    -0.061569
0.60225     1.0008     1.0008     1341
    {'b_kappa_Intercept'       }      1.6706    0.29903       1.0794
2.291     1.0034     1.0034     1822
    {'b_betai_ID1'             }    -0.29988      0.201      -0.7117
0.076664    0.99967    0.99967     1489
    {'b_betai_ID2'             }     0.36526    0.19551    -0.017381
0.75608     1.0002     1.0002     1600
    {'b_betai_ID3'             }    0.098498    0.19388     -0.28542
0.47473     0.9998     0.9998     1491
    {'b_betai_ID4'             }    -0.49669    0.19417     -0.90432
-0.14904     1.0016     1.0016     1406
    {'b_betai_ID5'             }     0.30845    0.20355     -0.10458
0.71145     1.0013     1.0013     1659
    {'softkappamu'             }      10.807     5.0009       3.9714
23.293     1.0009     1.0009     1215
    {'kappa_mu'                }      10.809     4.9984         3.99
23.293     1.0009     1.0009     1214
    {'sd_ID__kappa_Intercept'  }     0.42584    0.40025     0.013131
1.3614     1.0007     1.0007     1038
    {'r_ID__kappa[1,Intercept]'}    -0.14319    0.35322      -1.0016
0.45636      1.004      1.004     1805
    {'r_ID__kappa[2,Intercept]'}     0.14982    0.36417     -0.51075
1.0319     1.0006     1.0006     2300
    {'r_ID__kappa[3,Intercept]'}    0.086053    0.34451     -0.55839
0.90607     1.0016     1.0016     2098
    {'r_ID__kappa[4,Intercept]'}    0.093164    0.34798     -0.54522
0.9182     1.0006     1.0006     2275
    {'r_ID__kappa[5,Intercept]'}    -0.23501    0.38774      -1.1246
0.36474     1.0019     1.0019     1548

b_kappa_x 95% CI: -0.49 to 4.52
cor_ID__kappa_Intercept__kappa_x 95% CI: -0.68 to 0.99
```

Parameter recovery is quite successful for κ and κ_μ, given the minimal sample size. For `b_kappa_x` there is substantial posterior density for both increases and decreases in concentration in the treatment condition (95% CI above), and the range of estimates for `kappa_mu_x` largely overlaps with that for `kappa_mu` in the control condition. The correlation between individual differences in concentration in the control and in their response to treatment also has substantial posterior density for both correlation and anti-correlation, so there is no clear evidence of a relationship between variation in the two conditions (correctly, since none was simulated).

Test hypothesis of treatment effect

```
looHdNull = looUnwrap(fitHdNull, optsSlow);
looHdKappa = looUnwrap(fitHdKappa, optsSlow);
looHdTreat = looUnwrap(fitHdTreat, optsSlow);
lcHd = looCompare({'model_hd_null', 'model_hd_kappa', 'model_hd_treatment'},
...
```

```
    {looHdNull, looHdKappa, looHdTreat});
fprintf('ELPD difference (mu & kappa - kappa): %s\n', elpdDiff(looHdTreat,
looHdKappa));
fprintf('ELPD difference (mu & kappa - null): %s\n', elpdDiff(looHdTreat,
looHdNull));

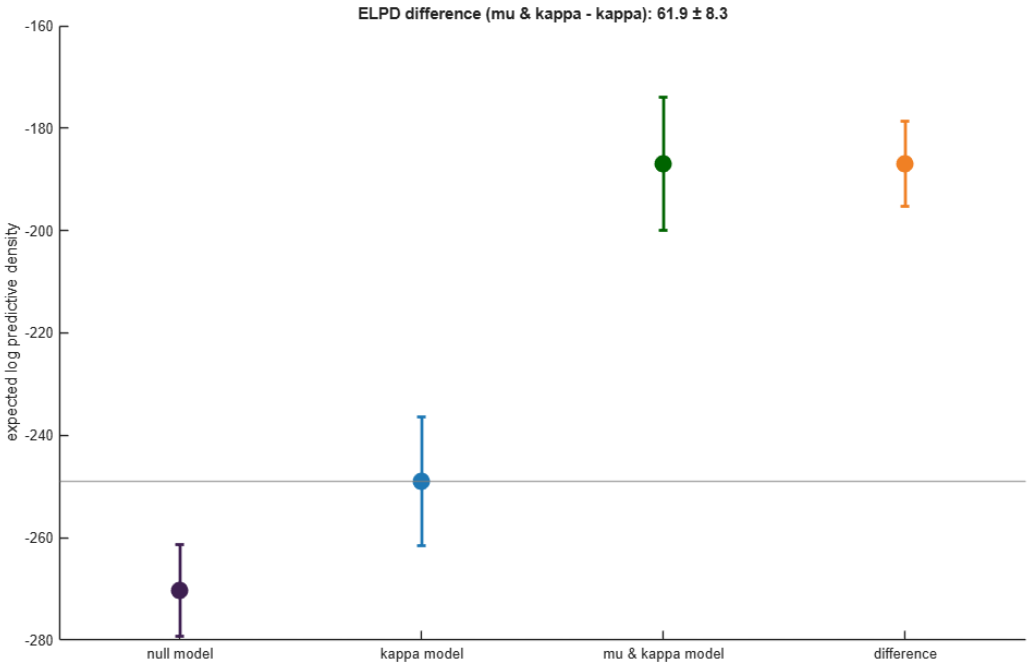
elpdPlot = [looHdNull.elpd, looHdKappa.elpd, looHdTreat.elpd];
sePlot = [looHdNull.se, looHdKappa.se, looHdTreat.se];
figure('Name', 'Model comparison', 'Color', 'w'); hold on;
cols = [col.obs; col.kappa; col.treat];
for i = 1:3
    errorbar(i, elpdPlot(i), sePlot(i), 'o', 'Color', cols(i, :), ...
        'MarkerFaceColor', cols(i, :), 'MarkerSize', 10, 'LineWidth', 2);
end
% difference between treatment and kappa models, plotted relative to the
kappa model
diffTK = looHdTreat.elpd - looHdKappa.elpd;
seTK = seDiff(looHdTreat, looHdKappa);
errorbar(4, looHdKappa.elpd + diffTK, seTK, 'o', 'Color', col.rho, ...
    'MarkerFaceColor', col.rho, 'MarkerSize', 10, 'LineWidth', 2);
yline(looHdKappa.elpd, 'Color', [0.5 0.5 0.5]);
xticks(1:4); xticklabels({'null model', 'kappa model', 'mu & kappa model',
'difference'});
xlim([0.5 4.5]); ylabel('expected log predictive density');
title(sprintf('ELPD difference (mu & kappa - kappa): %.1f ± %.1f', diffTK,
seTK));
saveFig(saveFigures, figDir, 'hd_model_comparison');
```

```
model_hd_kappa: 3 observation(s) with Pareto k > 0.7, refitting without them
Fitting model_hd_kappa without obs 44 (22 parameters)... 195 s, acceptance
ratio [0.99 0.97 0.99 0.99]
Fitting model_hd_kappa without obs 52 (22 parameters)... 132 s, acceptance
ratio [0.99 0.99 0.99 0.99]
Fitting model_hd_kappa without obs 136 (22 parameters)... 250 s, acceptance
ratio [0.99 0.98 0.99 0.99]

LOO comparison
            Model              elpd_diff    se_diff     elpd      se_elpd
    ___________________________    _________    _______    _______    _______

    {'model_hd_treatment'}           0           0      -186.98    12.943
    {'model_hd_kappa'    }      -61.904      8.2513     -248.88    12.576
    {'model_hd_null'     }      -83.275      10.903     -270.25    8.9302

ELPD difference (mu & kappa - kappa): 61.9 ± 8.3
ELPD difference (mu & kappa - null): 83.3 ± 10.9
```

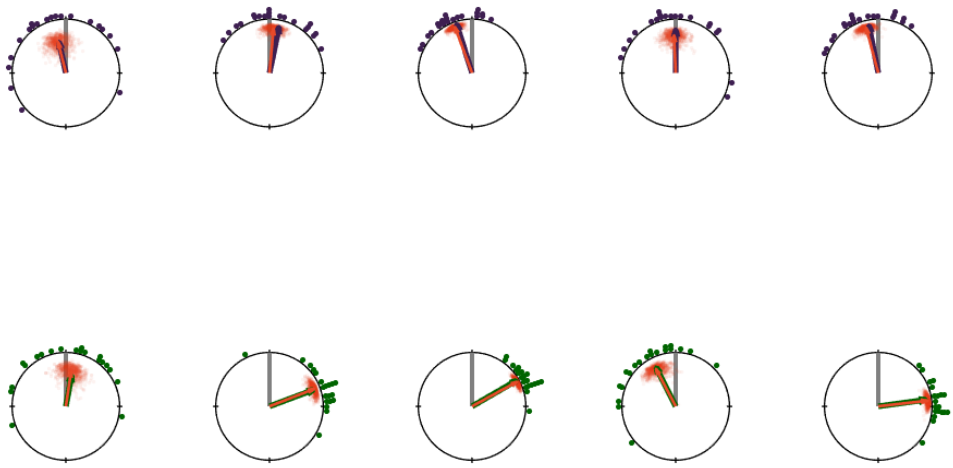


The results of model comparison are quite clear: the model for the hypothesis of an effect of treatment on both μ and κ predicts the data more robustly than the other two hypotheses.

Plot model predictions

Predictions are calculated separately for each condition for each individual: control (top row) and treatment (bottom row).

```
plotHeadingChange(fitHdTreat, dtHd, trueHd, col);
saveFig(saveFigures, figDir, 'hd_predictions');
```

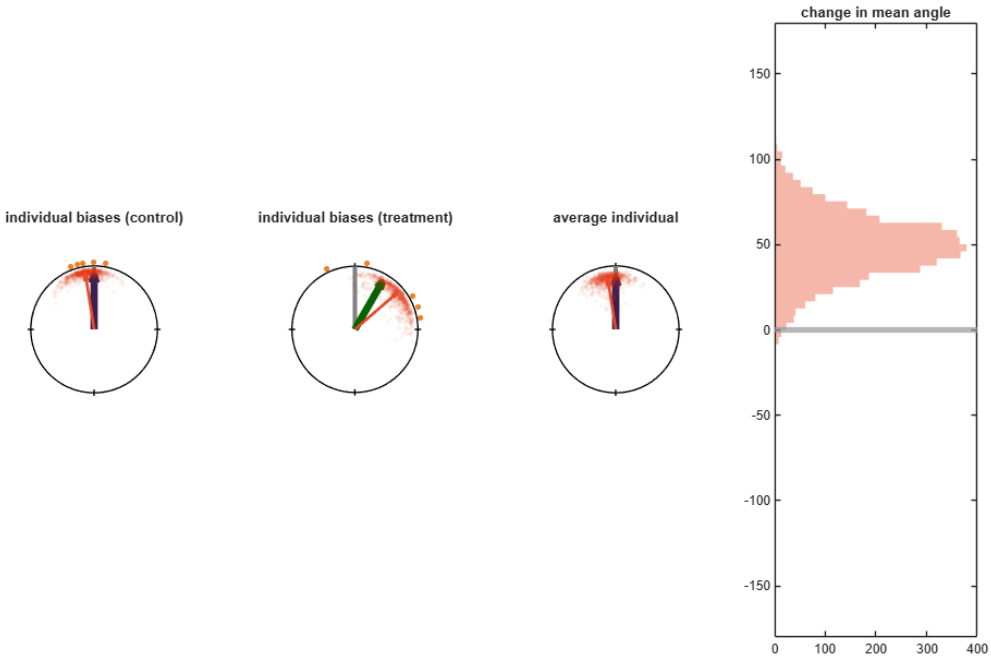


We can also compare the model parameters with the population parameters: individual biases in the control and treatment, the average individual, and the change in population mean angle.

```
b0 = getDraw(fitHdTreat, 'b_beta0_Intercept');
b0x = getDraw(fitHdTreat, 'b_beta0_x');
kmu = getDraw(fitHdTreat, 'kappa_mu');
kmux = getDraw(fitHdTreat, 'kappa_mu_x');
k0 = softplus(getDraw(fitHdTreat, 'b_kappa_Intercept'));
figure('Name', 'Population parameters', 'Color', 'w');
tiledlayout(1, 4);
ax = nexttile; circBase(ax, 0); title(ax, 'individual biases (control)');
circPoints(ax, deg2rad(trueHd.mu), col.rho);
circArrow(ax, 0, A1(4.0), col.obs, 5);
drawCloud(ax, b0, kmu);
circArrow(ax, circMean(b0), A1(median(kmu)), col.sd, 2);
ax = nexttile; circBase(ax, 0); title(ax, 'individual biases (treatment)');
circPoints(ax, deg2rad(trueHd.mu_delta), col.rho);
circArrow(ax, deg2rad(30), A1(4.0), col.treat, 5);
drawCloud(ax, b0x, kmux);
circArrow(ax, circMean(b0x), A1(median(kmux)), col.sd, 2);
ax = nexttile; circBase(ax, 0); title(ax, 'average individual');
circArrow(ax, 0, A1(3.0), col.obs, 5);
drawCloud(ax, b0, k0);
circArrow(ax, circMean(b0), A1(median(k0)), col.sd, 2);
ax = nexttile;
vertHist(ax, rad2deg(unwrapCircular(b0x)), 'change in mean angle', [-180
180], col.sd, 0);
saveFig(saveFigures, figDir, 'hd_population');
fprintf('Rightwards turn: %.1f%% of estimates; 95%% CI: %.0f° to %.0f° (true
```

```
turn 30°)\n', ...
    100*mean(wrapPi(b0x) > 0), quantile(rad2deg(unwrapCircular(b0x)), [0.025
0.975]));
```

Rightwards turn: 99.4% of estimates; 95% CI: 10° to 88° (true turn 30°)



The estimates of the population mean angle favour a right turn (percentage above), but there is a lot of uncertainty about the exact size of the turn. This is influenced by the wide spread of individual mean angles and turn angles, which the model recovered well.

Summary

- Circular GLMs can be used to model effects on mean direction μ and concentration κ.
- The posterior density of parameter estimates can be compared with expected values; for example, an expected angle (such as the 'home direction') or concentration.
- Where expected values of μ and κ are not known, fitted estimates can be compared between data for control and treatment conditions as an offset parameter between the two.
- Grouping factors, such as individual biases, can be accounted for in a hierarchical model, and the properties of the population of individuals estimated as hyperparameters for the concentration of individual mean directions (κ_μ) and standard deviation of individual concentrations (σ_κ).
- Competing hypotheses about effects within a dataset can be formalised as circular GLMs with differing parameters, and their plausibility contrasted directly via model-comparison methods such as expected log predictive density.

Published with MATLAB® R2026a